

Cell-to-Cell Communication Through the Substrate Only: The No-Direct-Channels Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 2 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one specific architectural commitment that constrains the design space for cell-to-cell coordination — that all coordination state passing from one cell's output to another cell's behavior flows through the substrate, with no direct channels permitted between cells.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's substrate-cell boundary commits to a small set of admissible boundary crossings; cell-to-cell direct flows are not among them. This note formalizes the implication as its own architectural commitment: in CKS, all coordination state that flows from one cell's output to another cell's behavior passes through the substrate, regardless of how technically convenient a direct channel might be. The note states the commitment in four operational components, distinguishes it from six adjacent technical patterns commonly conflated with it, identifies four downstream architectural properties for which the commitment is load-bearing, enumerates eight failure modes that violate it, and provides an operational test. The note closes the substrate-cell-boundary decomposition begun in four prior notes: any system claiming CKS-coherence on the boundary axis must satisfy all five commitments together.

1. Why the no-direct-channels commitment needs to be formalized as standalone

The parent foundational note on the substrate-cell boundary commits to the boundary as a joint architectural property. The four prior notes in this decomposition treat what each layer commits to alone, what crosses the boundary in each direction, and how cell-internal state differs from substrate state. The boundary-crossing treatment names direct cell-to-cell flows as one of the forbidden classes. This note formalizes that forbidden class as its own standalone commitment, with particular weight on why it is load-bearing for downstream properties.

The commitment needs standalone treatment because the implementation pressure toward direct channels is consistent and strong. Multi-cell systems where cell A produces output that cell B needs immediately, where cells could share memory in-process, or where cells coordinate frequently enough that substrate-mediated communication looks unnecessarily indirect all push toward direct exchanges. The architecture forbids such exchanges, and naming the reasons explicitly is what makes the commitment defensible rather than stylistic.

Two architectural reasons most directly ground it. Substrate-as-source-of-truth requires that authoritative coordination state live in the substrate; state flowing directly between cells leaves the substrate without the record of what was communicated, by whom, to whom, and when. Path retraceability requires that coordination paths be reconstructible from substrate provenance; a direct call from cell A to cell B leaves no substrate trace, with the path $A \rightarrow B$ existing in the runtime but invisible to anyone exercising the inspect right. Standalone formalization provides architectural vocabulary that addresses both concerns at once and makes anti-patterns identifiable without re-deriving them from the parent commitments each time.

2. The commitment, defined precisely

In the CKS pattern, cell-to-cell communication is **CKS-coherent** if and only if all coordination state that flows from one cell's output to another cell's behavior passes through the substrate. The commitment has four operational components.

(a) Cell A's outputs become substrate content. When cell A produces output that may influence other cells, the output is committed to the substrate as substrate content via a cell $\rightarrow$ substrate write under orchestration rules, with the provenance the substrate's design requires (writer, timestamp, rule, antecedents, rationale). The output is not held as cell-internal state to be passed elsewhere through some other channel.

(b) Cell B reads substrate content. When cell B's behavior depends on cell A's output, cell B reads the relevant substrate content via a substrate $\rightarrow$ cell read within cell B's own bounded scope and orchestration rules. The read is performed against substrate state at the time cell B executes; the content cell B receives is whatever the substrate currently carries, which may include intervening human modifications or supersessions.

(c) The substrate is the only architectural path. There is no architectural provision for direct flows between cell A and cell B — no shared memory references that pass between them, no message-queue payloads carrying coordination state outside the substrate, no callback registrations from one cell into another, no RPC calls between cells with coordination state in request or response, no agent-framework agent-to-agent protocols carrying coordination payloads.

(d) The path is fully traceable. The complete path from cell A's contribution to cell B's behavior — cell A's write provenance, the substrate content the write produced, cell B's read, and cell B's subsequent writes — is reconstructible from substrate content alone. Any human exercising the inspect right can recover the coordination path without consulting runtime state, queues, application logs, or any non-substrate source.

The four components together define what makes cell-to-cell communication CKS-coherent. The commitment is *architectural*: it concerns where coordination state lives during cell-to-cell exchange, not message ordering, latency, or synchronization. A direct channel — even a low-latency synchronous one — violates the commitment if state passes between cells outside the substrate. A substrate-mediated exchange that has high latency, runs asynchronously, or batches its writes is fully CKS-coherent if the architectural pattern is preserved.

3. What the commitment does NOT forbid

Stating precisely what the commitment does not forbid is what keeps the framing from drifting into something stronger than the source paper supports.

Asynchrony is not forbidden. Cell A writes; cell B reads when its orchestration rules trigger execution. Latency between write and read is a deployment concern, not an architectural one. Batched cycles, parallel waves, and event-driven invocations are all admissible.

Shared execution infrastructure is not forbidden. Cells may share compute platforms, connection pools, orchestration runtimes, or workflow-engine scheduling. Execution-coordination concerns — scheduling, resource allocation, observability — live at the orchestration layer adjacent to CKS; the commitment concerns coordination state.

Shared substrate access is not forbidden. Multiple cells reading the same substrate content in parallel is fully CKS-coherent. Multiple cells writing to overlapping scopes is also admissible, with contradictions preserved as substrate state and resolved at the cell layer under orchestration rules.

Notification mechanisms are not forbidden. A workflow engine, message bus, or notification system may signal that cell A's write occurred and trigger cell B's execution. The signal sits at the orchestration layer; cell B's coordination state input still comes from the substrate read it performs. A signal of the form "substrate region X was updated by cell A at time T" is fine; a signal of the form "cell A produced this decision payload, now act on it" is a coordination-state transfer dressed as a signal, and falls under what the commitment forbids.

Adjacent-component flows are not forbidden. When a cell uses an external API, retrieval system, or adjacent AI component, the flow is cell-to-component, not cell-to-cell. It is governed by hybrid composition rules, not by this commitment.

4. What the commitment is NOT

Six adjacent technical patterns are commonly conflated with the no-direct-channels commitment. The pattern across all six is consistent: the commitment is not about whether the technical pattern exists in a deployment; it is about *what flows through it*.

Not a constraint on shared memory. A CKS deployment may implement substrate access through shared memory regions; the substrate's host environment may itself live in such regions, and cells may access shared substrate from multiple processes. What the commitment forbids is cell-to-cell state passing through shared memory *outside* the substrate's architectural identity, not shared memory in general.

Not a constraint on message queues. A queue that signals "cell A's write occurred" is fine; cell B's coordination state input comes from cell B's substrate read. A queue that carries "cell A's decision payload directly to cell B's input" is a direct channel and violates the commitment. The same queue infrastructure can be coherent (signal-only) or violating (payload-carrying) depending on what flows through it.

Not a constraint on event buses. Same logic as message queues: events that signal substrate changes are fine; events that carry coordination state payloads between cells are direct channels.

Not a constraint on RPC/gRPC. RPC is permissible for cell-to-component flows. RPC between cells with coordination state in the request or response payload is a direct channel and violates the commitment.

Not a constraint on callback chains. Cell A registering a callback into cell B's execution context, with coordination state in the callback payload, is a direct channel.

Not a constraint on agent-to-agent protocols. This subsection deserves the most weight because it is the most operationally distinctive contrast in current practice. Agent frameworks — modern multi-agent systems and their successors — typically define agent-to-agent communication as a first-class architectural feature: agents pass messages, decisions, intermediate reasoning, and tool results directly to each other through the framework's native channels. Inter-agent coordination is the framework's responsibility; a substrate, when present at all, is often a side-store rather than the primary coordination medium. When CKS cells are implemented through such frameworks, the framework's native agent-to-agent protocols are direct channels in the architectural sense. A CKS deployment using such frameworks must constrain agents to substrate-mediated communication only — typically by turning off, narrowly scoping, or signal-only-restricting the framework's native channels and routing all coordination state through substrate writes and reads. The framework is not forbidden; using its native channels to carry coordination state is. The commitment runs against the grain of how agent frameworks are typically designed and used; satisfying it requires architectural discipline the frameworks do not by default enforce.

5. Why the commitment is load-bearing

The commitment is load-bearing for four downstream architectural properties, each treated as a separate foundational commitment elsewhere in this series.

Source-of-truth. Substrate-mediated cell-to-cell communication ensures the substrate carries the full record of inter-cell coordination state. Direct channels would leave coordination state outside the substrate at exactly the points where one cell's output influences another's behavior. A system with direct channels has a substrate that is authoritative about state at rest but silent about state in transit; the source-of-truth commitment requires authority over both.

Path retraceability. Substrate-mediated flows produce traceable coordination paths through the provenance metadata the substrate's design requires. Direct channels produce paths invisible to substrate inspection. A reader exercising the inspect right can see what each cell wrote, but cannot reconstruct how cell A's output influenced cell B's behavior — the connecting path ran outside substrate.

Conflict preservation. When cell A and cell B both write to substrate and produce contradicting content, the contradiction is preserved as substrate state under the conflict-as-first-class commitment. When cell A passes state directly to cell B, the contradiction has nowhere to materialize: cell B receives whatever cell A's interpretation happens to be, and the substrate never sees the contradiction. Conflict preservation depends on cell outputs landing in substrate where they can be compared and contradictions made addressable.

Composition requirements. Multi-cell, multi-substrate, and cross-organizational compositions all depend on the no-direct-channels property. Each of these composition modes requires that the connecting path between cells be substrate; each is impossible if the connecting path is a direct channel.

The commitment is therefore the operational basis on which several CKS properties depend at the multi-cell scale. Removing it produces a system that fails several other commitments at exactly the points where multi-cell behavior matters most.

6. Failure modes that violate the commitment

Each of the following is a way an implementation can introduce direct cell-to-cell channels.

(a) Shared in-memory state between cells. Cells running in the same process sharing state through global variables, shared objects, or in-process caches scoped across multiple cells.

(b) Direct cell invocation with payload. Cell A invokes cell B as a function, RPC, or service call with coordination state in the payload.

(c) Callback registration with payload. Cell A registers a callback that cell B invokes with coordination state in the callback payload.

(d) Message-queue payloads carrying coordination state. Messages between cells carry coordination state — decisions, rationale, intermediate outputs — rather than signals about substrate changes.

(e) Event-bus payloads carrying coordination state. Same pattern: events carry coordination state rather than signaling substrate changes.

(f) Workflow engine variable passing. A workflow engine passes variables between workflow steps that happen to be CKS cells. The workflow engine should signal cell triggering; coordination state should flow through substrate writes and reads, not through workflow variables.

(g) Agent-framework agent-to-agent protocols. When CKS cells are implemented through agent frameworks and agents communicate directly via the framework's native protocols, the protocols are direct channels for coordination state. This failure mode is the most prevalent in current multi-agent practice because frameworks treat agent-to-agent channels as a first-class feature; using a framework as a CKS substrate-coordination layer rather than its native agent-to-agent layer requires actively constraining the framework's defaults. The risk is that an implementation team adopts the framework, uses it idiomatically, and produces a system that looks like multi-cell coordination but routes its coordination state through framework-native channels rather than substrate. Visible signs include substrate writes that record only final outputs rather than the intermediate coordination state cells exchanged, and substrate inspection that cannot reconstruct who said what to whom.

(h) "Optimization" shortcuts. When implementations introduce shortcuts to bypass substrate writes and reads for performance reasons — cell A's output cached and passed directly to cell B without committing to substrate first; intermediate results held in process memory across cells "to avoid the round trip" — the cache or in-memory state becomes a direct channel. This failure mode is particularly insidious because it slips past architectural review under "performance" framing: described as avoiding redundant substrate writes, it is in fact a structural change that bypasses substrate for a class of inter-cell flows. A useful diagnostic: if removing the optimization changes the inspectable substrate content available to a human exercising the inspect right, the optimization was not optimizing — it was bypassing.

7. Operational test

A multi-cell system satisfies the no-direct-channels commitment if and only if all of the following are true at all times:

1. Every cell→substrate write that may influence other cells is committed to substrate state, with full provenance, as a cell→substrate boundary crossing.
2. Every substrate→cell read that supplies coordination state to a cell's behavior is performed by the receiving cell against substrate state, as a substrate→cell boundary crossing.

3. No coordination state passes from one cell to another through any mechanism that bypasses the substrate — not shared memory, not message queues, not event buses, not RPC, not callbacks, not agent-framework agent-to-agent protocols, not workflow variables, not optimization shortcuts.
4. Signaling mechanisms between cells (workflow triggers, queue messages, event notifications) carry only signals about substrate state, not coordination state payloads themselves.
5. The complete coordination path from cell A's contribution to cell B's behavior is reconstructible from substrate content alone, without consulting runtime state, queues, application logs, or any non-substrate source.

A system that fails any of (1)–(5) has direct cell-to-cell channels somewhere in its multi-cell coordination, even if subtly. Such a system may function and may exhibit some CKS-adjacent properties; it is not CKS-coherent on the cell-to-cell axis, and the source-of-truth, retraceability, conflict-preservation, and composition properties depending on the commitment fail wherever the direct channels run.

8. Why naming this commitment as standalone matters

Implementations under pressure to support multi-cell coordination consistently drift toward direct channels because direct channels are technically simpler, lower-latency, and aligned with familiar patterns from agent frameworks and microservice architectures. Substrate-mediated coordination requires architectural discipline the natural implementation pressures resist; without explicit commitment to no-direct-channels, that discipline tends to erode under deadline. The result is systems where multi-cell coordination becomes invisible to substrate inspection — the system functions, but the substrate stops being the source of truth at exactly the moments when multi-cell behavior matters most. Failure modes accumulate as auditability gaps, governance failures, and conflict-handling errors, each appearing as a separate downstream defect rather than a consequence of the architectural drift that produced them.

With this note complete and its companions on the substrate side standalone, the cell side standalone, the boundary crossings, and the cell-internal vs. substrate state distinction, the substrate-cell-boundary decomposition is fully formalized as prior art. The substrate side commits to four properties; the cell side commits to four properties; the boundary permits three classes of crossings, forbids four, and generates provenance at writes; the state distinction places coordination state in the substrate and execution state in cells; and cell-to-cell coordination flows exclusively through the substrate. Any system claiming CKS-coherence on the substrate-cell boundary axis must satisfy all five commitments together. Subsequent work that adopts, extends, composes, or argues against the CKS pattern should use the no-direct-channels commitment in the sense formalized here. Subsequent work that uses cell-to-cell coordination in a different sense is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Cell-to-Cell Communication Through the Substrate Only: The No-Direct-Channels Commitment in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.